ITSAR2 313 – SYSTEM INTEGRATION AND ARCHICTECTURE 2
LABORATORY REPORT: EXERCISE 1 MONOLITH VS MICROSERVICE

Delicano, Rovin Destajo; Herman, Angel Mae Arnaiz; Olmedo, Jirah Canillo; Prado, Jhona Mae Octavio; Sudayon, Karyl Ann Delmita
Information Technology, College of Computer Studies
Carlos Hilado Memorial State University

Abstract

This laboratory demonstrates the implementation of a three-layer architecture in a Product Ordering API using Python and Flask. The system separates concerns into a presentation layer, a business logic layer, and a data layer. Each layer has a distinct responsibility, business rules including product existence validation, quantity constraints, and stock availability checks are enforced through the business logic layer and tested using curl commands. The results confirm that enforcing rules at the logic layer rather than in the API routes or data layer produces a maintainable, integrity-preserving system that correctly handles both valid orders and edge case violations.

I. INTRODUCTION

Business logic refers to the set of rules, validations, and workflows that govern how data is created, modified, and processed within a software system. It serves as the decision-making layer of an application, sitting between the presentation layer that handles user interaction and the data layer that manages storage and retrieval. In modern software architecture, separating these concerns is a fundamental principle that improves maintainability, testability, and data integrity.

In the context of an order processing system, business logic enforces constraints such as: a product must exist before it can be ordered; the requested quantity must be a positive integer; and the available stock must be sufficient to fulfill the order. Without these rules enforced at a dedicated layer, invalid states can propagate into the database, leading to data corruption and unpredictable system behavior.

This exercise builds a RESTful API using Python and the Flask microframework. The system architecture follows a strict layered model: (1) the presentation layer in server.py handles HTTP routing and response formatting; (2) the business logic layer in layers/business_logic.py contains all order validation rules; and (3) the data layer in layers/data_layer.py manages the in-memory product database. The API is tested using curl, a command-line tool for making HTTP requests, which validates each business rule through a series of edge case tests.

II. METHODOLOGY / IMPLEMENTATION

Business logic refers to the set of rules, validations, and workflows that govern how data is created, modified, and processed within a software system. It serves as the decision-making layer of an application, sitting between the presentation layer that handles user interaction and the data layer that manages storage and retrieval. In modern software architecture, separating these concerns is a fundamental principle that improves maintainability, testability, and data integrity.

In the context of an order processing system, business logic enforces constraints such as: a product must exist before it can be ordered; the requested quantity must be a positive integer; and the available stock must be sufficient to fulfill the order. Without these rules enforced at a dedicated layer, invalid states can propagate into the database, leading to data corruption and unpredictable system behavior.

This exercise builds a RESTful API using Python and the Flask microframework. The system architecture follows a strict layered model: (1) the presentation layer in server.py handles HTTP routing and response formatting; (2) the business logic layer in layers/business_logic.py contains all order validation rules; and (3) the data layer in

layers/data_layer.py manages the in-memory product database. The API is tested using curl, a command-line tool for making HTTP requests, which validates each business rule through a series of edge case tests.

III. EXPERIMENTAL FINDINGS / OBSERVATIONS

The following edge cases were tested using curl with the -i flag to display HTTP response headers alongside the JSON response body. The results are summarized in the table below:

Test Case	Expected Result
Invalid product ID	400 BAD_REQUEST — Product with ID does not exist
Quantity = 0	400 BAD_REQUEST — Quantity must be greater than zero
Negative quantity	400 BAD_REQUEST — Quantity must be greater than zero
Stock exceeded	409 CONFLICT — Insufficient stock for product
Valid order	200 OK — Order successful, stock updated

GET all products

```
C:\WINDOWS\system32>curl -i -X GET http://localhost:5000/products
HTTP/1.1 200 OK
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 18 Mar 2026 13:43:57 GMT
Content-Type: application/json
Content-Length: 542
Connection: close

{
  "products": [
    {
      "id": "P001",
      "name": "Laptop",
      "price": 49999.0,
      "stock": 5
    },
    {
      "id": "P002",
      "name": "Mechanical Keyboard",
      "price": 3499.0,
      "stock": 10
    },
    {
      "id": "P003",
      "name": "USB-C Hub",
      "price": 1299.0,
      "stock": 0
    },
    {
      "id": "P004",
      "name": "Monitor",
      "price": 15999.0,
      "stock": 3
    },
    {
      "id": "P005",
      "name": "Webcam",
      "price": 2799.0,
      "stock": 7
    }
  ],
  "total": 5
}

C:\WINDOWS\system32>
```

GET single product

```
C:\WINDOWS\system32>curl -i -X GET http://localhost:5000/products/P001
HTTP/1.1 200 OK
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 18 Mar 2026 13:45:00 GMT
Content-Type: application/json
Content-Length: 73
Connection: close

{
  "id": "P001",
  "name": "Laptop",
  "price": 49999.0,
  "stock": 5
}
```

POST product (invalid)

```
C:\WINDOWS\system32>curl -i -X POST http://localhost:5000/orders -H "Content-Type: application/json" -d "{\"product_id\": \"X999\", \"quantity\": 1}"
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 18 Mar 2026 13:52:02 GMT
Content-Type: application/json
Content-Length: 86
Connection: close

{
  "error": "404 NOT_FOUND",
  "message": "Product with ID 'X999' does not exist."
}

C:\WINDOWS\system32>
```

POST product (0 quantity)

```
C:\WINDOWS\system32>curl -i -X POST http://localhost:5000/orders -H "Content-Type: application/json" -d "{\"product_id\": \"P001\", \"quantity\": 0}"
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 18 Mar 2026 13:52:54 GMT
Content-Type: application/json
Content-Length: 126
Connection: close

{
  "error": "400 BAD_REQUEST",
  "message": "Quantity must be greater than zero. You cannot order zero or negative items."
}
```

POST product (negative quantity)

```
C:\WINDOWS\system32>curl -i -X POST http://localhost:5000/orders -H "Content-Type: application/json" -d "{\"product_id\": \"P001\", \"quantity\": -5}"
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 18 Mar 2026 13:53:20 GMT
Content-Type: application/json
Content-Length: 126
Connection: close

{
  "error": "400 BAD_REQUEST",
  "message": "Quantity must be greater than zero. You cannot order zero or negative items."
}
```

POST product (insufficient)

```
C:\WINDOWS\system32>curl -i -X POST http://localhost:5000/orders -H "Content-Type: application/json" -d "{\"product_id\": \"P001\", \"quantity\": 999}"
HTTP/1.1 409 CONFLICT
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 18 Mar 2026 13:53:42 GMT
Content-Type: application/json
Content-Length: 109
Connection: close

{
  "error": "409 CONFLICT",
  "message": "Insufficient stock for 'Laptop'. Requested: 999, Available: 5."
}
```

POST product (valid)

```
C:\WINDOWS\system32>curl -i -X POST http://localhost:5000/orders -H "Content-Type: application/json" -d "{\"product_id\": \"P001\", \"quantity\": 2}"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 18 Mar 2026 13:55:23 GMT
Content-Type: application/json
Content-Length: 149
Connection: close

{
  "message": "Order successful",
  "orderedQty": 2,
  "product": "Laptop",
  "remainingStock": 3,
  "totalCost": 99998.0,
  "unitPrice": 49999.0
}
```

GET single product (update)

```
C:\WINDOWS\system32>curl -i -X GET http://localhost:5000/products/P001
HTTP/1.1 200 OK
Server: Werkzeug/3.1.6 Python/3.14.3
Date: Wed, 18 Mar 2026 13:55:59 GMT
Content-Type: application/json
Content-Length: 73
Connection: close

{
  "id": "P001",
  "name": "Laptop",
  "price": 49999.0,
  "stock": 3
}
```

All five test cases produced the expected results. The invalid product ID test returned a 404 NOT FOUND status with a descriptive error message. Both the zero-quantity and negative-quantity tests returned 400 BAD REQUEST responses, confirming that the quantity validation rule correctly rejects non-positive values. The stock exceeded test returned a 409 CONFLICT, indicating that the stock availability rule was enforced. The valid order test returned a 200 OK response with a JSON body confirming the product name, ordered quantity, unit price, total cost, and remaining stock after deduction.

A follow-up GET request to /products/P001 after the valid order confirmed that the stock value had been decremented, verifying that the data layer update was applied correctly. The separation of concerns was observed to function as intended: the server routes contained no validation logic, and the data layer was only modified after all business rules had passed.

IV. LAB DISCUSSIONS / COMPUTATIONS

The results of the curl tests confirm that the layered architecture enforces business rules consistently across all request types. The use of a custom BusinessRuleViolation exception class allowed the business logic layer to communicate both the error message and the HTTP status code to the presentation layer without the routes needing to contain any decision-making logic.

One notable observation is the distinction between HTTP status codes: 400 BAD REQUEST is used for malformed or logically invalid input (zero or negative quantity), while 409 CONFLICT is used for valid input that cannot be fulfilled due to the current state of the system (insufficient stock). This distinction reflects REST API design conventions and provides more precise feedback to the client.

The in-memory data store demonstrates the role of the data layer cleanly but is not persistent — restarting the server resets all stock values. In a production system, this layer would be replaced with a database-backed implementation. Because the data layer is isolated, this replacement would require no changes to the business logic or presentation layers, demonstrating the practical benefit of the layered architecture.

Guide Questions and Answers

1. What is business logic in software architecture?
Business logic is the decision-making layer of an application that defines and enforces rules on how data is created, modified, and validated before it is processed or stored.

2. Why should business logic not be placed in the UI layer?
UI-based rules can be bypassed by directly calling the API, and would need to be duplicated across every client. A dedicated logic layer ensures all requests are validated in one place regardless of where they come from.

3. How does business logic improve data integrity?
It validates every request before it reaches the data layer, preventing invalid entries such as negative stock or nonexistent products from ever being written to the database.

4. What happens if business logic is not implemented?
The system will accept any request without validation, allowing invalid orders to go through, stock to go negative, and corrupted data to accumulate over time.

V. CONCLUSIONS

This laboratory activity successfully demonstrated the implementation and testing of a business logic layer within a three-tier API architecture. By isolating order validation rules in a dedicated layer, the system achieved clear separation of concerns: the presentation layer handled only HTTP routing, the business logic layer made all decisions about order validity, and the data layer performed only read and write operations without any knowledge of business constraints.

The curl-based edge case tests confirmed that each business rule — product existence, quantity validity, and stock sufficiency — was enforced correctly and that appropriate HTTP status codes were returned for each scenario. The valid order test additionally confirmed that the stock deduction was applied to the data layer only after all rules had passed.

The exercise reinforced the importance of placing business logic at the correct architectural layer. Had these rules been embedded in the API routes or the data layer, the system would be harder to test, maintain, and extend. The layered approach ensures that any future changes to the rules, the data store, or the API interface can be made independently without affecting the other layers.

References

Flask Documentation. (2024). Flask — A Python Microframework. Pallets Projects. https://flask.palletsprojects.com

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley Professional.

Python Software Foundation. (2024). Python 3 Documentation. https://docs.python.org/3/

Richardson, L., & Ruby, S. (2007). RESTful Web Services. O'Reilly Media.

Mozilla Developer Network. (2024). HTTP Response Status Codes. https://developer.mozilla.org/en-US/docs/Web/HTTP/Status